

University of Calgary | Department of Computer Science
CPSC 471: Database Management Systems | Winter 2026

BrightCore Merch Store

An Internal Merchandise Order & Fulfillment System
Final Project Report | Group G-32

Name	Student ID
Shazil Khan	30241942
Tameem Aboueldahab	30204890
Surkhab Mundi	30189028

April 2026

Abstract

BrightCore Merch Store is a relational database-driven internal merchandise ordering and fulfillment system designed for the BrightCore organization. The system replaces a previously manual, spreadsheet-based workflow with a centralized platform that manages product listings, inventory, employee orders, payments, and fulfillment operations in one place. The database is implemented in MySQL and exposed through a Django REST API consumed by a React frontend.

The system supports three distinct user roles: Employee, FulfillmentAdmin, and ProductAdmin. Employees browse a product catalog, select variants by size and color, place orders, and track their order status. FulfillmentAdmins record payments, update order statuses, and complete the fulfillment workflow. ProductAdmins manage the catalog, inventory levels, and user accounts.

The schema consists of thirteen entity types including one weak entity (Variant) and over ten relationship types, with full referential integrity enforced through primary keys, foreign keys, unique constraints, and NOT NULL constraints. Critical workflows such as order placement are executed as atomic database transactions to guarantee data consistency. Email notifications are automatically logged on every order status change through a Django signal handler, maintaining a complete and auditable communication trail.

1. Introduction

1.1 Problem Statement

BrightCore is an organization that distributes branded merchandise to its employees as part of its internal operations. Before this project, merchandise orders were managed manually using spreadsheets and informal communication channels. This approach led to inaccurate inventory counts, incomplete order records, and no reliable mechanism for tracking an order through its full lifecycle from placement to fulfillment. As order volume grew, these limitations became increasingly difficult to manage and introduced real financial and logistical risk.

From a database perspective, this scenario surfaces the core challenges central to this course: maintaining referential integrity across a network of related entities, enforcing business rules such as preventing stock from going negative, supporting atomic multi-step workflows, and producing reliable reporting data for administrators.

1.2 Proposed Solution

The system addresses these problems by consolidating catalog management, inventory tracking, order placement, payment recording, and fulfillment into a single platform with clearly defined user roles and enforced data constraints. MySQL provides the relational foundation, Django and Django REST Framework form the application layer, and React serves as the client interface. All critical workflows are transaction-safe, and data integrity is enforced at both the application and database levels.

2. Project Design

2.1 System Architecture

The BrightCore Merch Store follows a standard three-tier architecture separating the client, application, and data layers.

Tier	Technology	Responsibility
Client Layer	React + Vite + Tailwind CSS	Employee and admin-facing web interface
Application Layer	Django + Django REST Framework	Business logic, REST API, JWT authentication
Data Layer	MySQL 8.0 (InnoDB)	Schema, constraints, transactions, data persistence

The frontend communicates with the backend exclusively through REST API calls. JWT tokens issued at login carry the user's role (employee, fulfillment_admin, or product_admin) embedded in the payload, which both the frontend and backend use to enforce role-based access control.

2.2 User Roles and Responsibilities

Employee

Employees are the standard end users of the system. After logging in, they can browse the product catalog, view full product details, select a specific size and color combination, and place an order. Order placement is executed as an atomic database transaction: the Order record and all OrderLine records are created and variant stock is decremented in a single operation. If any step fails, the entire transaction is rolled back. Employees can also view their complete order history and receive automatic email log entries on every status change.

FulfillmentAdmin

FulfillmentAdmins manage the operational side of order fulfillment. From their dashboard they can view all system orders filtered by status or date, record payments against orders (which automatically advances the order to Processing), update order statuses, and set pickup-ready dates and tracking numbers. Each payment is associated with the FulfillmentAdmin's ID for full traceability. They can also review the EmailLog table to confirm that notifications were delivered.

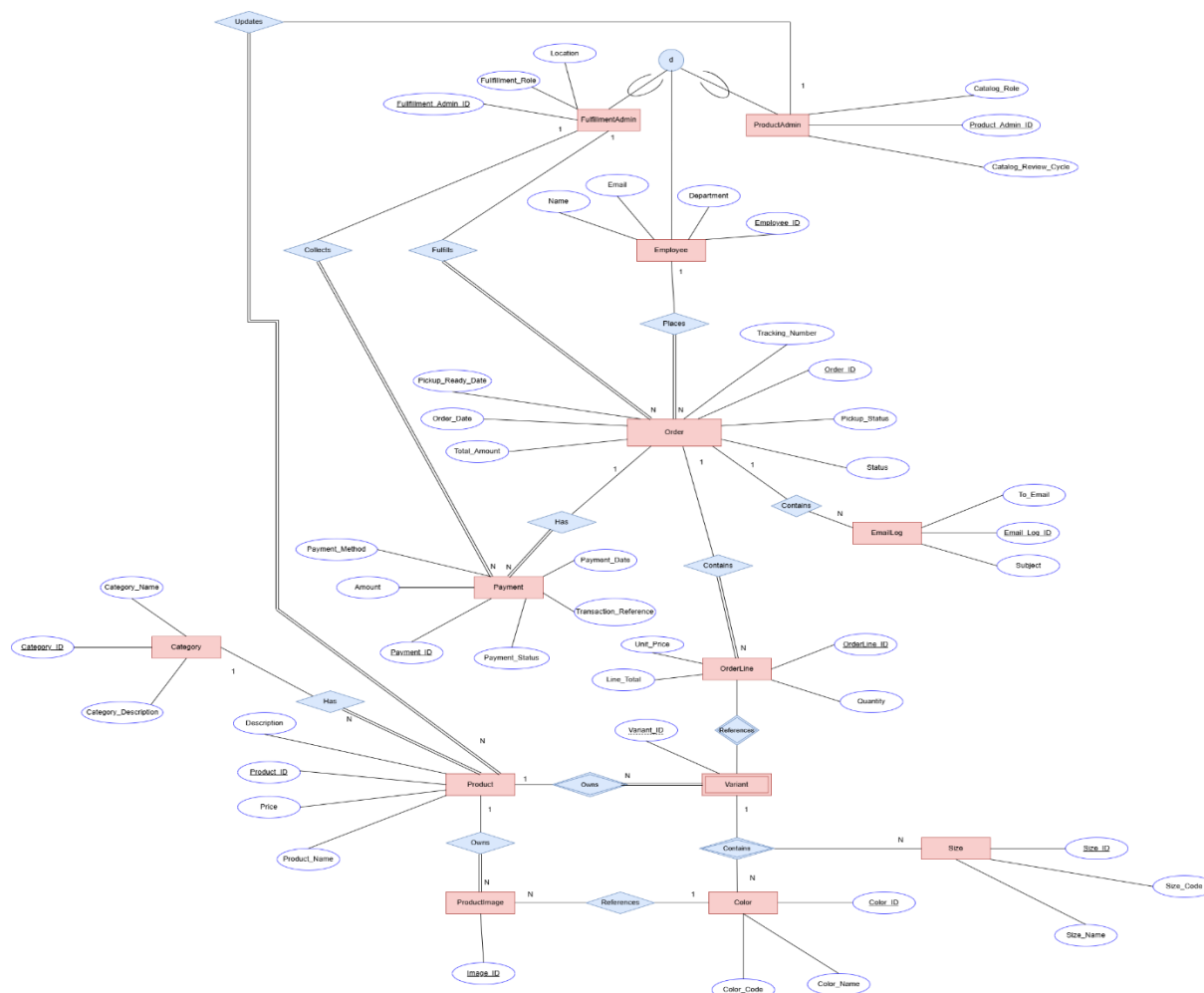
ProductAdmin

ProductAdmins manage the product catalog. They can create, update, and delete categories and products, manage size and color combinations per product, set and adjust stock quantities per variant, and upload product images. Each product is linked to the ProductAdmin responsible for it. ProductAdmins also manage employee and admin accounts, including promoting employees to either admin role and deactivating existing ones.

2.3 EERD Overview

The Enhanced Entity-Relationship Diagram for the BrightCore Merch Store was refined through two iterations based on TA feedback. The key changes applied were: adding more complete attributes to entities, removing redundant Employee_ID attributes from subclass tables, and replacing the original Admin and SuperAdmin roles with the operationally specific ProductAdmin and FulfillmentAdmin roles.

The design contains thirteen entity types, satisfying the requirement for at least eight unique entities. Employee is the supertype; FulfillmentAdmin and ProductAdmin are disjoint partial subtypes. Variant is the weak entity, owned by Product through an identifying relationship. The full entity list is: Employee, FulfillmentAdmin, ProductAdmin, Order, OrderLine, Product, Category, Variant (weak), Size, Color, Payment, EmailLog, and ProductImage.



2.4 Relationship Summary

Relationship	Cardinality	Participation
--------------	-------------	---------------

Employee Places Order	1 Employee : N Orders	Total on Order side
Order Contains OrderLine	1 Order : N OrderLines	Total on OrderLine side
OrderLine References Variant	N OrderLines : 1 Variant	Total on OrderLine side
Category Has Product	1 Category : N Products	Total on Product side
Product Identifies Variant	1 Product : N Variants	Total on Variant (weak)
Product Owns ProductImage	1 Product : N ProductImages	Partial on ProductImage
Size Applies to Variant	1 Size : N Variants	Total on Variant side
Color Applies to Variant	1 Color : N Variants	Total on Variant side
Order Has Payment	1 Order : 1 Payment	Partial on Payment side
Order Has EmailLog	1 Order : N EmailLogs	Partial on EmailLog side
Color Referenced by ProductImage	1 Color : N ProductImages	Partial (nullable FK)

2.5 Key Design Assumptions

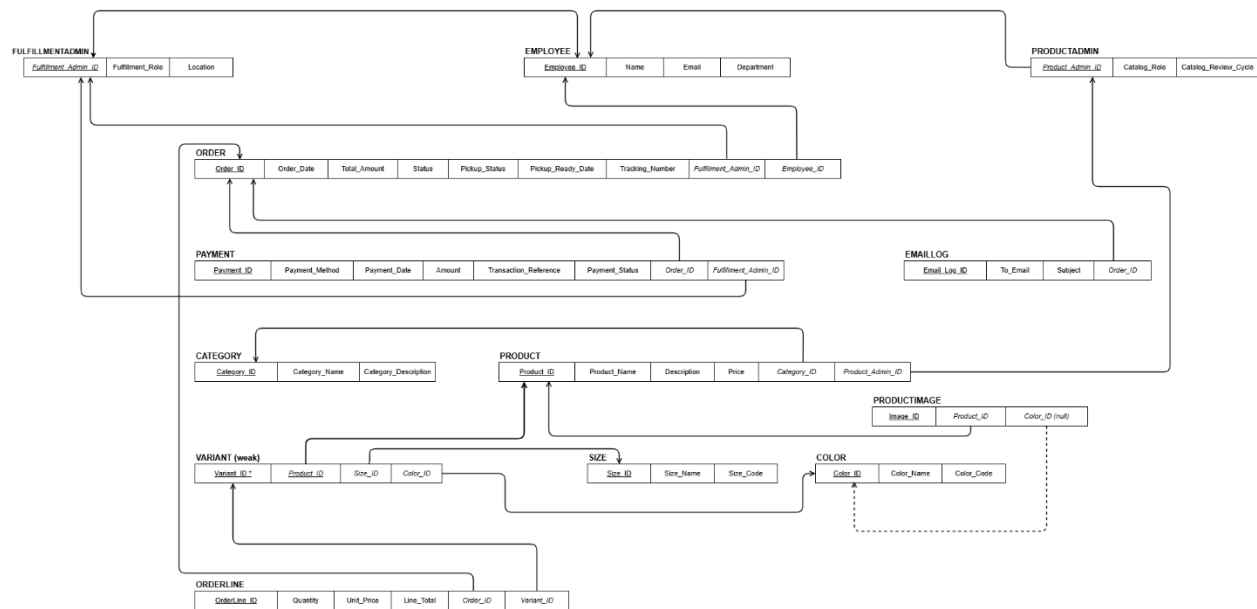
- Employee is the supertype for all users. A regular employee without an admin record can browse and place orders but cannot access any admin workflows.
- The FulfillmentAdmin and ProductAdmin specializations are disjoint and partial. An Employee cannot simultaneously hold both admin roles.
- Variant is a weak entity fully dependent on Product. Each Variant represents a unique (Product, Size, Color) combination enforced by a composite UNIQUE constraint.
- Each Order has at most one Payment, enforced by a UNIQUE constraint on Order_ID in the PAYMENT table.
- Email log entries are created automatically on every order status transition, providing a complete and immutable audit trail.
- ProductImages may optionally reference a Color. If no Color is specified, the image is treated as a generic product image visible for all color variants.
- Deleting a Product cascades to its Variants and ProductImages. Deleting an Order cascades to its OrderLines, Payment, and EmailLog entries.

3. Implementation

3.1 Relational Schema

The following schema was derived from the EERD using the standard ERD-to-relational mapping algorithm. Each entity type maps to a relation, and relationship types are represented through foreign key attributes.

Relation	Primary Key	Foreign Keys / Constraints
EMPLOYEE	Employee_ID	Email UNIQUE, Password NOT NULL
FULLFILLMENTADMIN	Fulfillment_Admin_ID	Employee_ID FK + UNIQUE (1:1 to EMPLOYEE)
PRODUCTADMIN	Product_Admin_ID	Employee_ID FK + UNIQUE (1:1 to EMPLOYEE)
CATEGORY	Category_ID	Category_Name UNIQUE
PRODUCT	Product_ID	Category_ID FK (RESTRICT), Product_Admin_ID FK (RESTRICT)
SIZE	Size_ID	Size_Code UNIQUE
COLOR	Color_ID	Color_Code UNIQUE
VARIANT (weak)	Variant_ID	Product_ID FK (CASCADE), Size_ID FK, Color_ID FK; UNIQUE(Product_ID, Size_ID, Color_ID)
PRODUCTIMAGE	Image_ID	Product_ID FK (CASCADE), Color_ID FK nullable (SET NULL)
ORDER	Order_ID	Employee_ID FK (RESTRICT), Fulfillment_Admin_ID FK nullable; Status ENUM
PAYMENT	Payment_ID	Order_ID FK (CASCADE) + UNIQUE, Fulfillment_Admin_ID FK nullable; Transaction_Reference UNIQUE
ORDERLINE	OrderLine_ID	Order_ID FK (CASCADE), Variant_ID FK (RESTRICT)
EMAILLOG	Email_Log_ID	Order_ID FK (CASCADE); Sent_At DEFAULT CURRENT_TIMESTAMP



Notable Mapping Decisions

- The disjoint partial specialization of Employee was mapped using separate subtype tables. Each subtype table references EMPLOYEE with a foreign key and a UNIQUE constraint, enforcing the one-to-one nature of the relationship.
- Variant uses a composite UNIQUE KEY on (Product_ID, Size_ID, Color_ID) to reflect its partial key from the EERD, preventing duplicate variant combinations per product.
- The one-to-one relationship between Order and Payment is enforced by a UNIQUE constraint on Order_ID in PAYMENT rather than merging the tables, preserving clean separation and allowing the payment record to remain absent until a FulfillmentAdmin processes it.
- ON DELETE RESTRICT on PRODUCT prevents a Category from being deleted while it still contains products. The same rule applies to Variant, preventing deletion if it appears in any OrderLine, preserving historical order accuracy.

3.2 DBMS Selection

MySQL 8.0 with the InnoDB storage engine was selected as the DBMS. InnoDB provides row-level locking, foreign key constraint enforcement, and full ACID transaction support, all of which are required for the order placement workflow. Django's ORM connects to MySQL through the mysqlclient adapter, and Django migrations generate and apply the DDL. All tables are created with utf8mb4 encoding to support the full Unicode character set.

3.3 SQL DDL — CREATE TABLE Statements

Tables are created in dependency order to satisfy all foreign key constraints. The ORDER table name is backtick-quoted because ORDER is a reserved word in MySQL.

```
CREATE DATABASE IF NOT EXISTS brightcore_merch;
USE brightcore_merch;
```



```

-- 1. EMPLOYEE
CREATE TABLE EMPLOYEE (
    Employee_ID INT NOT NULL AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Department VARCHAR(50) NOT NULL,
    Password VARCHAR(255) NOT NULL,
    PRIMARY KEY (Employee_ID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 2. FULFILLMENTADMIN
CREATE TABLE FULFILLMENTADMIN (
    Fulfillment_Admin_ID INT NOT NULL AUTO_INCREMENT,
    Employee_ID INT NOT NULL UNIQUE,
    Fulfillment_Role VARCHAR(50) NOT NULL,
    Location VARCHAR(100) NOT NULL,
    PRIMARY KEY (Fulfillment_Admin_ID),
    CONSTRAINT fk_fa_emp FOREIGN KEY (Employee_ID)
        REFERENCES EMPLOYEE(Employee_ID) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 3. PRODUCTADMIN
CREATE TABLE PRODUCTADMIN (
    Product_Admin_ID INT NOT NULL AUTO_INCREMENT,
    Employee_ID INT NOT NULL UNIQUE,
    Catalog_Role VARCHAR(50) NOT NULL,
    Catalog_Review_Cycle VARCHAR(50) NOT NULL,
    PRIMARY KEY (Product_Admin_ID),
    CONSTRAINT fk_pa_emp FOREIGN KEY (Employee_ID)
        REFERENCES EMPLOYEE(Employee_ID) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 4. CATEGORY
CREATE TABLE CATEGORY (
    Category_ID INT NOT NULL AUTO_INCREMENT,
    Category_Name VARCHAR(100) NOT NULL UNIQUE,
    Category_Description TEXT,
    PRIMARY KEY (Category_ID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 5. PRODUCT
CREATE TABLE PRODUCT (
    Product_ID INT NOT NULL AUTO_INCREMENT,
    Product_Name VARCHAR(150) NOT NULL,
    Description TEXT,
    Price DECIMAL(10,2) NOT NULL,
    Category_ID INT NOT NULL,
    Product_Admin_ID INT NOT NULL,
    PRIMARY KEY (Product_ID),
    CONSTRAINT fk_prod_cat FOREIGN KEY (Category_ID)
        REFERENCES CATEGORY(Category_ID) ON DELETE RESTRICT,
    CONSTRAINT fk_prod_pa FOREIGN KEY (Product_Admin_ID)
        REFERENCES PRODUCTADMIN(Product_Admin_ID) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 6. SIZE
CREATE TABLE SIZE (
    Size_ID INT NOT NULL AUTO_INCREMENT,
    Size_Name VARCHAR(50) NOT NULL,
    Size_Code VARCHAR(10) NOT NULL UNIQUE,
    PRIMARY KEY (Size_ID)
)

```

```

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 7. COLOR
CREATE TABLE COLOR (
  Color_ID INT NOT NULL AUTO_INCREMENT,
  Color_Name VARCHAR(50) NOT NULL,
  Color_Code VARCHAR(7) NOT NULL UNIQUE,
  PRIMARY KEY (Color_ID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 8. VARIANT (weak entity)
CREATE TABLE VARIANT (
  Variant_ID INT NOT NULL AUTO_INCREMENT,
  Product_ID INT NOT NULL,
  Size_ID INT NOT NULL,
  Color_ID INT NOT NULL,
  Stock_Quantity INT NOT NULL DEFAULT 0,
  PRIMARY KEY (Variant_ID),
  UNIQUE KEY uq_variant (Product_ID, Size_ID, Color_ID),
  CONSTRAINT fk_var_prod FOREIGN KEY (Product_ID)
    REFERENCES PRODUCT(Product_ID) ON DELETE CASCADE,
  CONSTRAINT fk_var_size FOREIGN KEY (Size_ID)
    REFERENCES SIZE(Size_ID) ON DELETE RESTRICT,
  CONSTRAINT fk_var_color FOREIGN KEY (Color_ID)
    REFERENCES COLOR(Color_ID) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 9. PRODUCTIMAGE
CREATE TABLE PRODUCTIMAGE (
  Image_ID INT NOT NULL AUTO_INCREMENT,
  Product_ID INT NOT NULL,
  Color_ID INT DEFAULT NULL,
  Image_URL VARCHAR(500) NOT NULL,
  Alt_Text VARCHAR(200),
  PRIMARY KEY (Image_ID),
  CONSTRAINT fk_img_prod FOREIGN KEY (Product_ID)
    REFERENCES PRODUCT(Product_ID) ON DELETE CASCADE,
  CONSTRAINT fk_img_color FOREIGN KEY (Color_ID)
    REFERENCES COLOR(Color_ID) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 10. ORDER
CREATE TABLE `ORDER` (
  Order_ID INT NOT NULL AUTO_INCREMENT,
  Order_Date DATE NOT NULL,
  Total_Amount DECIMAL(10,2) NOT NULL,
  Status ENUM('Pending','Processing',
    'Ready_for_Pickup','Fulfilled','Cancelled')
    NOT NULL DEFAULT 'Pending',
  Pickup_Status VARCHAR(50) DEFAULT NULL,
  Pickup_Ready_Date DATE DEFAULT NULL,
  Tracking_Number VARCHAR(100) DEFAULT NULL,
  Employee_ID INT NOT NULL,
  Fulfillment_Admin_ID INT DEFAULT NULL,
  PRIMARY KEY (Order_ID),
  CONSTRAINT fk_ord_emp FOREIGN KEY (Employee_ID)
    REFERENCES EMPLOYEE(Employee_ID) ON DELETE RESTRICT,
  CONSTRAINT fk_ord_fa FOREIGN KEY (Fulfillment_Admin_ID)
    REFERENCES FULFILLMENTADMIN(Fulfillment_Admin_ID) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```
-- 11. PAYMENT
CREATE TABLE PAYMENT (
    Payment_ID INT NOT NULL AUTO_INCREMENT,
    Payment_Method VARCHAR(50) NOT NULL,
    Payment_Date DATE NOT NULL,
    Amount DECIMAL(10,2) NOT NULL,
    Transaction_Reference VARCHAR(150) NOT NULL UNIQUE,
    Payment_Status ENUM('Pending','Completed','Refunded')
        NOT NULL DEFAULT 'Pending',
    Order_ID INT NOT NULL UNIQUE,
    Fulfillment_Admin_ID INT DEFAULT NULL,
    PRIMARY KEY (Payment_ID),
    CONSTRAINT fk_pay_ord FOREIGN KEY (Order_ID)
        REFERENCES `ORDER`(Order_ID) ON DELETE CASCADE,
    CONSTRAINT fk_pay_fa FOREIGN KEY (Fulfillment_Admin_ID)
        REFERENCES FULFILLMENTADMIN(Fulfillment_Admin_ID) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 12. ORDERLINE
CREATE TABLE ORDERLINE (
    OrderLine_ID INT NOT NULL AUTO_INCREMENT,
    Quantity INT NOT NULL,
    Unit_Price DECIMAL(10,2) NOT NULL,
    Line_Total DECIMAL(10,2) NOT NULL,
    Order_ID INT NOT NULL,
    Variant_ID INT NOT NULL,
    PRIMARY KEY (OrderLine_ID),
    CONSTRAINT fk_ol_ord FOREIGN KEY (Order_ID)
        REFERENCES `ORDER`(Order_ID) ON DELETE CASCADE,
    CONSTRAINT fk_ol_var FOREIGN KEY (Variant_ID)
        REFERENCES VARIANT(Variant_ID) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 13. EMAILLOG
CREATE TABLE EMAILLOG (
    Email_Log_ID INT NOT NULL AUTO_INCREMENT,
    To_Email VARCHAR(100) NOT NULL,
    Subject VARCHAR(200) NOT NULL,
    Body TEXT NOT NULL,
    Sent_At DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    Order_ID INT NOT NULL,
    PRIMARY KEY (Email_Log_ID),
    CONSTRAINT fk_log_ord FOREIGN KEY (Order_ID)
        REFERENCES `ORDER`(Order_ID) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

3.4 Stored Procedures

The following stored procedures encapsulate the core business transactions of the system. They represent the logic implemented in the Django backend expressed as native MySQL stored procedures, demonstrating atomic transaction handling with error recovery.

Procedure 1 — Place Order (Atomic Transaction)

This is the most critical transaction in the system. It creates an Order record, inserts OrderLine records, decrements variant stock, and computes the order total — all within a single atomic transaction. If any step fails, the entire transaction is rolled back.

```

DELIMITER $$
CREATE PROCEDURE PlaceOrder(
    IN p_employee_id INT,
    IN p_variant_id INT,
    IN p_quantity INT,
    OUT p_order_id INT,
    OUT p_result VARCHAR(100)
)
BEGIN
    DECLARE v_stock INT;
    DECLARE v_price DECIMAL(10,2);
    DECLARE v_total DECIMAL(10,2);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_result = 'ERROR: Transaction rolled back';
    END;

    START TRANSACTION;

    SELECT v.Stock_Quantity, p.Price
    INTO v_stock, v_price
    FROM VARIANT v
    JOIN PRODUCT p ON p.Product_ID = v.Product_ID
    WHERE v.Variant_ID = p_variant_id
    FOR UPDATE;

    IF v_stock < p_quantity THEN
        ROLLBACK;
        SET p_result = 'ERROR: Insufficient stock';
    ELSE
        SET v_total = v_price * p_quantity;

        INSERT INTO `ORDER` (Order_Date, Total_Amount, Status, Employee_ID)
        VALUES (CURDATE(), v_total, 'Pending', p_employee_id);

        SET p_order_id = LAST_INSERT_ID();

        INSERT INTO ORDERLINE
        (Quantity, Unit_Price, Line_Total, Order_ID, Variant_ID)
        VALUES
        (p_quantity, v_price, v_total, p_order_id, p_variant_id);

        UPDATE VARIANT
        SET Stock_Quantity = Stock_Quantity - p_quantity
        WHERE Variant_ID = p_variant_id;

        COMMIT;
        SET p_result = 'SUCCESS';
    END IF;
END$$
DELIMITER ;

```

Procedure 2 — Record Payment and Advance Order Status

Records a payment for an order and automatically advances the order status to Processing. Validates that no payment has already been recorded before proceeding.

```

DELIMITER $$
CREATE PROCEDURE RecordPayment(
    IN p_order_id INT,
    IN p_fulfillment_admin_id INT,
    IN p_method VARCHAR(50),
    IN p_amount DECIMAL(10,2),
    IN p_transaction_ref VARCHAR(150),
    OUT p_result VARCHAR(100)
)
BEGIN
    DECLARE v_exists INT DEFAULT 0;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN ROLLBACK; SET p_result = 'ERROR'; END;

    START TRANSACTION;

    SELECT COUNT(*) INTO v_exists
    FROM PAYMENT WHERE Order_ID = p_order_id;

    IF v_exists > 0 THEN
        ROLLBACK;
        SET p_result = 'ERROR: Payment already recorded';
    ELSE
        INSERT INTO PAYMENT
            (Payment_Method, Payment_Date, Amount,
             Transaction_Reference, Payment_Status,
             Order_ID, Fulfillment_Admin_ID)
        VALUES
            (p_method, CURDATE(), p_amount,
             p_transaction_ref, 'Completed',
             p_order_id, p_fulfillment_admin_id);

        UPDATE `ORDER`
        SET Status = 'Processing',
            Fulfillment_Admin_ID = p_fulfillment_admin_id
        WHERE Order_ID = p_order_id;

        COMMIT;
        SET p_result = 'SUCCESS';
    END IF;
END$$
DELIMITER ;

```

Procedure 3 — Full Order Summary Report

Retrieves a complete order summary for the FulfillmentAdmin dashboard, joining across seven tables to show employee details, product information, variant selection, and payment status.

```

SELECT
    o.Order_ID,
    o.Order_Date,
    e.Name AS Employee_Name,
    e.Department,
    p.Product_Name,

```

```

    s.Size_Code,
    c.Color_Name,
    ol.Quantity,
    ol.Unit_Price,
    ol.Line_Total,
    o.Total_Amount,
    o.Status,
    pay.Payment_Status,
    pay.Payment_Method
FROM `ORDER` o
JOIN EMPLOYEE e ON e.Employee_ID = o.Employee_ID
JOIN ORDERLINE ol ON ol.Order_ID = o.Order_ID
JOIN VARIANT v ON v.Variant_ID = ol.Variant_ID
JOIN PRODUCT p ON p.Product_ID = v.Product_ID
JOIN SIZE s ON s.Size_ID = v.Size_ID
JOIN COLOR c ON c.Color_ID = v.Color_ID
LEFT JOIN PAYMENT pay ON pay.Order_ID = o.Order_ID
ORDER BY o.Order_Date DESC;

```

Procedure 4 — Low Stock Report

Identifies all product variants with critically low inventory (at or below 5 units). Used by ProductAdmins to monitor stock levels and trigger restocking before items run out.

```

SELECT
    p.Product_Name,
    cat.Category_Name,
    s.Size_Code,
    c.Color_Name,
    v.Stock_Quantity
FROM VARIANT v
JOIN PRODUCT p ON p.Product_ID = v.Product_ID
JOIN CATEGORY cat ON cat.Category_ID = p.Category_ID
JOIN SIZE s ON s.Size_ID = v.Size_ID
JOIN COLOR c ON c.Color_ID = v.Color_ID
WHERE v.Stock_Quantity <= 5
ORDER BY v.Stock_Quantity ASC;

```

3.5 INSERT Statements — Sample Data

The following inserts populate the database with representative seed data used during development and demos. Data is inserted in dependency order.

```

INSERT INTO EMPLOYEE (Name, Email, Department, Password) VALUES
    ('Alice Johnson', 'alice@brightcore.com', 'Engineering', 'test123'),
    ('Bob Smith', 'bob@brightcore.com', 'Marketing', 'test123'),
    ('Carol White', 'carol@brightcore.com', 'Operations', 'test123'),
    ('David Lee', 'david@brightcore.com', 'HR', 'test123'),
    ('Emma Davis', 'emma@brightcore.com', 'Operations', 'test123');

INSERT INTO FULFILLMENTADMIN (Employee_ID, Fulfillment_Role, Location) VALUES
    (3, 'Senior Fulfillment', 'Warehouse A'),
    (5, 'Fulfillment Staff', 'Warehouse B');

```

```

INSERT INTO PRODUCTADMIN (Employee_ID, Catalog_Role, Catalog_Review_Cycle) VALUES
(4, 'Catalog Manager', 'Monthly');

INSERT INTO CATEGORY (Category_Name, Category_Description) VALUES
('Tops', 'T-shirts, hoodies, and polo shirts'),
('Accessories', 'Hats, bags, and other accessories'),
('Bottoms', 'Pants, shorts, and joggers');

INSERT INTO SIZE (Size_Name, Size_Code) VALUES
('Extra Small', 'XS'), ('Small', 'S'), ('Medium', 'M'),
('Large', 'L'), ('Extra Large', 'XL');

INSERT INTO COLOR (Color_Name, Color_Code) VALUES
('Black', '#000000'), ('White', '#FFFFFF'),
('Navy Blue', '#001F5B'), ('Grey', '#808080');

INSERT INTO PRODUCT (Product_Name, Description, Price, Category_ID, Product_Admin_ID)
VALUES
('BrightCore Classic Tee', 'Cotton t-shirt with BrightCore logo.', 24.99,
1, 1),
('BrightCore Hoodie', 'Warm pullover hoodie with embroidered logo.', 59.99,
1, 1),
('BrightCore Cap', 'Adjustable snapback cap.', 19.99,
2, 1),
('BrightCore Joggers', 'Comfortable fleece joggers.', 49.99,
3, 1);

INSERT INTO VARIANT (Product_ID, Size_ID, Color_ID, Stock_Quantity) VALUES
(1, 3, 1, 50), (1, 3, 2, 40), (1, 4, 1, 35),
(2, 3, 3, 20), (2, 4, 3, 15), (2, 5, 4, 10),
(3, 3, 1, 60), (3, 3, 3, 45),
(4, 3, 4, 30), (4, 4, 1, 25);

INSERT INTO PRODUCTIMAGE (Product_ID, Color_ID, Image_URL, Alt_Text) VALUES
(1, 1, '/media/products/tee-black.png', 'Classic Tee in Black'),
(1, 2, '/media/products/tee-white.png', 'Classic Tee in White'),
(2, 3, '/media/products/hoodie-navy-front.png', 'Hoodie Navy Front'),
(2, 3, '/media/products/hoodie-navy-back.png', 'Hoodie Navy Back'),
(2, 4, '/media/products/hoodie-grey-front.png', 'Hoodie Grey Front'),
(2, 4, '/media/products/hoodie-grey-back.png', 'Hoodie Grey Back'),
(3, 1, '/media/products/cap-black.png', 'Cap in Black'),
(3, 3, '/media/products/cap-navy.png', 'Cap in Navy Blue'),
(4, 4, '/media/products/joggers-grey.png', 'Joggers in Grey'),
(4, 1, '/media/products/joggers-black.png', 'Joggers in Black');

INSERT INTO `ORDER` (Order_Date, Total_Amount, Status, Employee_ID) VALUES
('2025-03-01', 24.99, 'Fulfilled', 1),
('2025-03-05', 79.98, 'Processing', 2),
('2025-03-10', 59.99, 'Pending', 1);

INSERT INTO PAYMENT
(Payment_Method, Payment_Date, Amount, Transaction_Reference, Payment_Status,
Order_ID)
VALUES
('Credit Card', '2025-03-01', 24.99, 'TXN-001-20250301', 'Completed', 1),
('Payroll Deduction', '2025-03-05', 79.98, 'TXN-002-20250305', 'Pending', 2);

INSERT INTO ORDERLINE (Quantity, Unit_Price, Line_Total, Order_ID, Variant_ID) VALUES
(1, 24.99, 24.99, 1, 1),

```

```
(1, 59.99, 59.99, 2, 4),
(1, 19.99, 19.99, 2, 7),
(1, 59.99, 59.99, 3, 5);
```

```
INSERT INTO EMAILLOG (To_Email, Subject, Body, Order_ID) VALUES
('alice@brightcore.com', 'Order #1 ready for pickup',
'Hi Alice, your order #1 is ready at Warehouse A.', 1),
('bob@brightcore.com', 'Order #2 is being processed',
'Hi Bob, your order #2 is currently being processed.', 2);
```

3.6 SELECT Statements

The following SELECT queries demonstrate the key read operations performed by the system.

Query 1 — Full order summary for the FulfillmentAdmin dashboard (joins seven tables):

```
SELECT
  o.Order_ID,
  o.Order_Date,
  e.Name      AS Employee_Name,
  e.Department,
  p.Product_Name,
  s.Size_Code,
  c.Color_Name,
  ol.Quantity,
  ol.Unit_Price,
  ol.Line_Total,
  o.Total_Amount,
  o.Status,
  pay.Payment_Status,
  pay.Payment_Method
FROM `ORDER` o
JOIN EMPLOYEE e ON e.Employee_ID = o.Employee_ID
JOIN ORDERLINE ol ON ol.Order_ID = o.Order_ID
JOIN VARIANT v ON v.Variant_ID = ol.Variant_ID
JOIN PRODUCT p ON p.Product_ID = v.Product_ID
JOIN SIZE s ON s.Size_ID = v.Size_ID
JOIN COLOR c ON c.Color_ID = v.Color_ID
LEFT JOIN PAYMENT pay ON pay.Order_ID = o.Order_ID
ORDER BY o.Order_Date DESC;
```

Query 2 — Retrieve all orders for a specific employee (My Orders page):

```
SELECT Order_ID, Order_Date, Total_Amount, Status,
       Pickup_Ready_Date, Tracking_Number
FROM `ORDER`
WHERE Employee_ID = 1
ORDER BY Order_Date DESC;
```


Query 3 — Low stock report (variants at or below 5 units):

```
SELECT p.Product_Name, cat.Category_Name,
       s.Size_Code, c.Color_Name, v.Stock_Quantity
FROM VARIANT v
JOIN PRODUCT p ON p.Product_ID = v.Product_ID
JOIN CATEGORY cat ON cat.Category_ID = p.Category_ID
JOIN SIZE s ON s.Size_ID = v.Size_ID
JOIN COLOR c ON c.Color_ID = v.Color_ID
WHERE v.Stock_Quantity <= 5
ORDER BY v.Stock_Quantity ASC;
```

Query 4 — Retrieve all email logs for a specific order:

```
SELECT Email_Log_ID, To_Email, Subject, Sent_At
FROM EMAILLOG
WHERE Order_ID = 1
ORDER BY Sent_At ASC;
```

3.7 UPDATE Statements

Update 1 — FulfillmentAdmin advances order status and sets fulfillment details:

```
UPDATE `ORDER`
SET     Status           = 'Ready_for_Pickup',
        Pickup_Ready_Date = '2025-03-15',
        Tracking_Number   = 'TRK-2025-0042'
WHERE   Order_ID = 2;
```

Update 2 — ProductAdmin restocks a variant:

```
UPDATE VARIANT
SET     Stock_Quantity = Stock_Quantity + 50
WHERE   Variant_ID = 4;
```

Update 3 — ProductAdmin updates a product price:

```
UPDATE PRODUCT SET Price = 27.99 WHERE Product_ID = 1;
```

Update 4 — FulfillmentAdmin marks a payment as Refunded:

```
UPDATE PAYMENT SET Payment_Status = 'Refunded' WHERE Payment_ID = 2;
```

3.8 DELETE Statements

Delete 1 — Cancel and remove an order (cascades to OrderLine, Payment, EmailLog):

```
-- Mark as Cancelled first to preserve the audit trail
UPDATE `ORDER` SET Status = 'Cancelled' WHERE Order_ID = 3;

-- Remove the record; cascade rule cleans up ORDERLINE, PAYMENT, EMAILLOG
DELETE FROM `ORDER` WHERE Order_ID = 3;
```

Delete 2 — Remove a product variant (restricted if it appears in any OrderLine):

```
-- Only succeeds if no OrderLine references this variant
DELETE FROM VARIANT WHERE Variant_ID = 9;
```

Delete 3 — Deactivate a FulfillmentAdmin account (Employee record is preserved):

```
DELETE FROM FULFILLMENTADMIN WHERE Fulfillment_Admin_ID = 2;
```

4. Visual Interface

4.1 Design Approach

The interface was designed with a focus on minimal typed input and intuitive navigation. The layout uses a persistent left sidebar for navigation, a clean white content area, and a neutral dark navy color scheme consistent with the BrightCore brand. All key interactions are selection-based, using dropdown menus, toggle buttons, and pre-populated fields wherever possible to reduce input errors. The frontend is built with React and Tailwind CSS, with routing handled by React Router and all API communication going through axios with JWT tokens.

4.2 Pages and Functionality

Login and Registration

The login page uses a two-column layout. The left panel displays the BrightCore logo and tagline on a dark navy background. The right panel contains the email and password form. On successful login, the server returns a JWT access token with the user's role embedded in the payload, which the frontend stores in React context. New employees can self-register through the Register link on the login page.

Product Catalog

After login, employees land on the catalog page showing all products in a card grid. Each card displays the product image with a hover effect that reveals the back of the product for items with multiple images (notably the hoodies). Cards are clickable and navigate to the product detail page. A search bar and category filter allow employees to find items quickly. Categories are fetched dynamically from the API.

Product Detail Page

The product detail page shows a full image gallery with previous and next arrows, dot indicators, and a thumbnail strip. Selecting a color updates the gallery to show images for that specific color. Size and color options are displayed as interactive buttons. Out-of-stock combinations are visually disabled. A stock badge shows In Stock, Low Stock, or Out of Stock. A quantity selector and Add to Cart button complete the interaction.

Cart and Checkout

The cart is accessible through a slide-in drawer from the sidebar. It shows all added items with product images, size, color, and quantity controls. The Place Order button creates the order record atomically in the database, then redirects to Stripe's secure checkout page. After payment, Stripe redirects back to the order confirmation page which records the payment in the database.

My Orders

The My Orders page displays the employee's full order history. Each order entry shows the order date, total, and a color-coded status badge. Clicking an order expands it to show the individual line items with product name, variant, quantity, and price.

Fulfillment Admin Dashboard

The FulfillmentAdmin dashboard shows all system orders in a table with status filter tabs and summary stat cards. Admins can update order status through a dropdown on each row. Recording a payment opens an inline form where the admin selects a payment method and confirms, which automatically advances the order to Processing and records the payment with the admin's ID attached.

Product Admin Dashboard

The ProductAdmin dashboard shows the full product catalog in a table with thumbnail images, category, price, and stock status. Admins can add, edit, and delete products through a modal form. A separate Manage Categories page provides the same create, edit, and delete functionality for categories.

5. User Guide

5.1 Getting Started

BrightCore Merch Store is a web-based application accessible through any modern browser. The Django backend must be running on localhost:8000 and the React frontend on localhost:5173. No client-side installation is required. New employees can register through the Register link on the login page.

5.2 Employee Workflow

1. Navigate to the application URL and click Login.
2. Enter your registered email and password and click Sign In.
3. Browse the Product Catalog on the home page. Click any product card to view its full details.
4. Select a color using the color swatches. The image gallery updates to show that color. Select a size using the size buttons. Out-of-stock combinations are shown as disabled.
5. Adjust the quantity and click Add to Cart. The cart icon in the sidebar shows the item count.
6. Click Cart in the sidebar to review your items, then click Checkout to proceed to payment.
7. Complete payment through Stripe. You will be redirected back to the order confirmation page.
8. Navigate to My Orders in the sidebar to track the status of your orders at any time.

5.3 FulfillmentAdmin Workflow

9. Log in with your FulfillmentAdmin credentials. You will be routed to the Fulfillment Dashboard.
10. Use the status filter tabs to view orders by status. The stat cards at the top show total counts.
11. To record a payment: click Record Payment on a Pending order, select the payment method, and click Confirm Payment. The order status advances to Processing automatically.
12. To update an order status: use the status dropdown on any order row and select the new status. An email log entry is created automatically on every status change.
13. For fulfilled orders, the pickup ready date and tracking number can be set through the order detail panel.

5.4 ProductAdmin Workflow

14. Log in with your ProductAdmin credentials. You will be routed to the Product Admin Dashboard.
15. To add a product: click Add Product, fill in the name, description, price, and category, then click Save Product.
16. To edit a product: click the pencil icon on any product row, update the fields, and save.
17. To delete a product: click the trash icon and confirm. Deletion is restricted if the product has order history.
18. Navigate to Manage Categories in the sidebar to create, edit, or delete product categories.

6. References

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

Django Software Foundation. (n.d.). Django documentation, version 5.2.
<https://docs.djangoproject.com/>

Django REST Framework. (n.d.). <https://www.django-rest-framework.org/>

Oracle Corporation. (n.d.). MySQL 8.0 reference manual. <https://dev.mysql.com/doc/refman/8.0/en/>

Meta Open Source. (n.d.). React documentation. <https://react.dev/>

Stripe, Inc. (n.d.). Stripe API documentation. <https://stripe.com/docs/api>

Shopify. (n.d.). Managing inventory. Shopify Help Center.
<https://help.shopify.com/en/manual/products/inventory>